Task 23: Apply Vibe Coding and Prompt Engineering techniques using a Generative AI tool. Design effective prompts to generate code snippets, cloud configuration templates, or application logic, and evaluate the quality and efficiency of the generated outputs for real-world business applications.

Aim

To apply **Vibe Coding** and **Prompt Engineering techniques** using a Generative AI tool (such as ChatGPT) to generate:

- Code snippets
- Cloud configuration templates
- Application logic

and evaluate their **quality, efficiency, and real-world usability**.

Problem Statement

Modern software development requires rapid development, automation, and scalability. Developers often spend significant time on:

- Writing boilerplate code
- Configuring cloud infrastructure
- Designing application logic

The challenge is:

Requirements

1. Tools & Technologies

- Generative AI Tool: ChatGPT / GitHub Copilot
- IDE: Spring Tool Suite / VS Code
- Programming Language: Java / Python / JavaScript
- Cloud Platform: Amazon Web Services (AWS) / Google Cloud Platform

2. Concepts Used

- Prompt Engineering
- Vibe Coding (intent-driven coding)
- AI-assisted development
- REST API development
- Cloud configuration (YAML/JSON)

3. Input Requirements

- Well-structured prompts
- Clear problem description
- Constraints (performance, scalability, etc.)

What is Vibe Coding?

- **Vibe Coding** = Writing prompts that express *intent, style, and constraints* rather than exact code.
- 👉 Example:
  Instead of:
- "Write Java code for REST API"
- Use:
- "Generate a clean, production-ready Spring Boot REST API with layered architecture, validation, exception handling, and pagination."

🛠️ Step-by-Step Implementation

🔹 Step 1: Define Use Case

Example Business Case:

Develop a **Student Management System API** with CRUD operations.

🔹 Step 2: Design Effective Prompts

✅ Prompt 1 – Code Generation

Generate a Spring Boot REST API for Student Management System with:
- Entity: Student (id, name, email)
- CRUD operations
- Layered architecture (Controller, Service, Repository)
- Use Spring Data JPA
- Include validation and exception handling
- Use pagination and sorting

✅ Prompt 2 – Cloud Configuration

Generate an AWS CloudFormation YAML template to deploy:
- EC2 instance
- Security group (allow HTTP, SSH)
- S3 bucket for storage
- Auto scaling configuration

✅ Prompt 3 – Application Logic

Write Python logic for:
- Predicting student performance using linear regression
- Include data preprocessing and evaluation metrics

🔹 Step 3: Generate Outputs Using AI

🔹 Step 4: Apply Vibe Coding Refinement

🔹 **Step 5: Evaluate Generated Outputs**

Criteria	Evaluation
Correctness	Code compiles and runs
Efficiency	Uses pagination, optimized queries
Readability	Clean structure
Scalability	Layered architecture
Security	Validation included

🔹 **Step 6: Compare Prompt Quality**

Prompt Type	Result
Basic Prompt	Generic code
Detailed Prompt	Production-ready code
Vibe Coding Prompt	Highly optimized & structured

📊 **Evaluation & Analysis**

✅ **Advantages**

- Faster development
- Reduced manual coding
- High-quality boilerplate generation
- Improves productivity

❌ **Limitations**

- Requires well-designed prompts
- May generate incorrect logic sometimes
- Needs validation/testing

🏢 **Real-World Business Applications**

- SaaS product development
- Cloud infrastructure automation
- AI-based analytics systems
- Enterprise application development

✅ Conclusion

By applying **Prompt Engineering + Vibe Coding** using tools like ChatGPT:

- Development time is reduced significantly
- Code quality improves with better prompts
- AI becomes a **collaborative coding partner**